Quasi-Newton Methods: From Rank-1 Update to Rank-k Update, From Convex Optimization to More.

Chengchang Liu

Joint works with Shuxian Bi, Cheng Chen, John C.S. Lui and Luo Luo

Outline

1. Introduction
2. Symmetric Rank-k Methods
3. Squared-Quasi-Newton Methods
4. Partial-Quasi-Newton Methods

1 Introduction

2 Symmetric Rank-k Methods

3 Squared-Quasi-Newton Methods

4 Partial-Quasi-Newton Methods

Why Quasi-Newton Methods?

Consider the minimization problems

$$\min_{\mathbf{x}} f(\mathbf{x}) \triangleq \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x}), \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^d$ and $\mu\mathbf{I} \preceq \nabla^2 f(\mathbf{x}) \preceq L\mathbf{I}$ and $\nabla^2 f(\mathbf{x})$ is L_2-Lipschitz Continuous ($\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\| \leq L_2\|\mathbf{x} - \mathbf{x}'\|$).

$$d, \quad \kappa = \frac{L}{\mu}, \quad n, \quad M = \frac{L_2}{\mu^{3/2}}$$

For the simplicity, we set $d \approx n$.

Why Quasi-Newton Methods?

We consider the following iteration to solve (1)

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1} \nabla f(\mathbf{x}_t).$$

- gradient Methods: $\mathbf{G}_t := \frac{1}{\eta_t}\mathbf{I}$, where η_t is the stepsize.
- Newton Methods: $\mathbf{G}_t := \nabla^2 f(\mathbf{x}_t)$

Order	flops per iteration	Convergence
first-order	$\mathcal{O}(d^2)$	linear ($\lim_{t\to\infty} \lambda_{t+1}/\lambda_t < 1$)
second-order	$\mathcal{O}(d^3)$	quadratic ($\lambda_{t+1}/\lambda_t = \lambda_t$)

Why Quasi-Newton Methods?

- gradient Methods: $\mathbf{G}_t := \frac{1}{\eta_t}\mathbf{I}$, where η_t is the stepsize.
- Newton Methods: $\mathbf{G}_t := \nabla^2 f(\mathbf{x}_t)$
- Quasi-Newton Methods :$\mathbf{G}_t := \mathbf{G}_{t-1} + \Delta_t$, where $\mathrm{rank}\Delta_t = 1, 2$.

Order	flops per iteration	Convergence
first-order	$\mathcal{O}(d^2)$	linear $(\lim_{t\to\infty} \lambda_{t+1}/\lambda_t < 1)$
second-order	$\mathcal{O}(d^3)$	quadratic $(\lambda_{t+1}/\lambda_t = \lambda_t)$
Quasi-Newton	$\mathcal{O}(d^2)$	superlinear $(\lim_{t\to\infty} \lambda_{t+1}/\lambda_t = 0)$

History of Quasi-Newton Methods

If we have seen further,
it is by standing on ye shoulders of giants.

History of Quasi-Newton Methods

- (1960s) DFP method (Fletcher, Powel): the first quasi-Newton method
- (1970s) Broyden's family method/BFGS/SR1 (C.G.Broyden): asymptotic superlinear convergence analysis
- (1980s) block BFGS (R.B. Schnabel): quasi-Newton methods with block update
- (1990s) adaptiveBFGS (Yuan): global convergent guarantee
- (2010s) randomized Block BFGS (Gower and Richtárik): use Hessian vector product to proceed the update.
- (2020s) greedy quasi-Newton (Rodomanov and Nesterov): non-asympototic analysis of quasi-Newton methods ...

Quasi-Newton Methods for Convex Optimization

Consider the iteration of quasi-Newton

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1} \nabla f(\mathbf{x}_t) \\ \mathbf{G}_{t+1} = \mathrm{Update}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t), \end{cases}$$

where $\mathbf{u}_t \in \mathbb{R}^{d \times 1}$, which satisfies

$$\mathbf{G}_{t+1} \mathbf{u}_t = \mathbf{H}_{t+1} \mathbf{u}_t.$$

Take BFGS udpate for example:

$$\mathrm{BFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) \triangleq \mathbf{G}_t - \frac{\mathbf{G}_t \mathbf{u}_t \mathbf{u}_t^\top \mathbf{G}_t}{\mathbf{u}_t^\top \mathbf{G}_t \mathbf{u}_t} + \frac{\mathbf{H}_{t+1} \mathbf{u}_t \mathbf{u}_t^\top \mathbf{H}_{t+1}}{\mathbf{u}_t^\top \mathbf{H}_{t+1} \mathbf{u}_t}$$

Quasi-Newton Methods for Convex Optimization

The classical quasi-Newton methods (1970s):

- We implement update by

$$\mathbf{u}_t = \mathbf{x}_{t+1} - \mathbf{x}_t \qquad \text{and} \qquad \mathbf{H}_{t+1} = \int_0^1 \nabla^2 f(\mathbf{x}_t + s\mathbf{u}_t)\,\mathrm{d}s,$$

which leads to $\underbrace{\mathbf{G}_{t+1}\mathbf{u}_t = \mathbf{H}_{t+1}\mathbf{u}_t = \nabla f(\mathbf{x}_{t+1}) - \nabla f(\mathbf{x}_t) \triangleq \mathbf{v}_t}_{\text{secant equation}}$.

- For example, the classical BFGS can be written as

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1}\nabla f(\mathbf{x}_t) \\ \mathbf{G}_{t+1} = \mathrm{BFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) = \mathbf{G}_t - \dfrac{\mathbf{G}_t\mathbf{u}_t\mathbf{u}_t^\top\mathbf{G}_t}{\mathbf{u}_t^\top\mathbf{G}_t\mathbf{u}_t} + \dfrac{\mathbf{v}_t\mathbf{v}_t^\top}{\mathbf{v}_t^\top\mathbf{u}_t}. \end{cases}$$

Quasi-Newton Methods for Convex Optimization

Greedy and Random Quasi-Newton Methods (2010-2020):

- 1. greedy quasi-Newton (Ronomanov and Nesterov(2020))

$$\mathbf{u}_t = \underset{\mathbf{u}\in\{\mathbf{e}_1,\ldots,\mathbf{e}_d\}}{\arg\max} \frac{\mathbf{u}^\top \mathbf{G}_t \mathbf{u}}{\mathbf{u}^\top \mathbf{H}_{t+1} \mathbf{u}}, \qquad \mathbf{H}_{t+1} = \nabla^2 f(\mathbf{x}_{t+1})$$

  2. Randomized quasi-Newton (Gower and Richtárik(2016), Lin et.al(2021))

$$\mathbf{u}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad \mathbf{H}_{t+1} = \nabla^2 f(\mathbf{x}_{t+1}).$$

  They satisfies $\mathbf{G}_{t+1}\mathbf{u}_t = \mathbf{H}_{t+1}\mathbf{u}_t = \underbrace{\nabla^2 f(\mathbf{x}_{t+1})\mathbf{u}_t}_{\text{Hessian-Vector Product}}$.

- For example, the greedy/random BFGS can be written as

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1}\nabla f(\mathbf{x}_t) \\ \tilde{\mathbf{G}}_t = (1 + M\|\mathbf{x}_{t+1} - \mathbf{x}_t\|_{\mathbf{x}_t})\mathbf{G}_t \\ \mathbf{G}_{t+1} = \tilde{\mathbf{G}}_t - \dfrac{\tilde{\mathbf{G}}_t \mathbf{u}_t \mathbf{u}_t^\top \tilde{\mathbf{G}}_t}{\mathbf{u}_t^\top \tilde{\mathbf{G}}_t \mathbf{u}_t} + \dfrac{\nabla^2 f(\mathbf{x}_{t+1})\mathbf{u}_t \mathbf{u}_t^\top \nabla^2 f(\mathbf{x}_{t+1})}{\mathbf{u}_t^\top \nabla^2 f(\mathbf{x}_{t+1})\mathbf{u}_t}. \end{cases}$$

Quasi-Newton Methods for Convex Optimization

Block Randomized quasi-Newton (R.B.Schnabel, Gower and Richtárik(2016), Kovalev et.al)

- BFGS update, choose $\mathbf{u}_t \in \mathbb{R}^{d\times 1}$

$$\mathrm{BFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) \triangleq \mathbf{G}_t - \frac{\mathbf{G}_t\mathbf{u}_t\mathbf{u}_t^\top\mathbf{G}_t}{\mathbf{u}_t^\top\mathbf{G}_t\mathbf{u}_t} + \frac{\mathbf{H}_{t+1}\mathbf{u}_t\mathbf{u}_t^\top\mathbf{H}_{t+1}}{\mathbf{u}_t^\top\mathbf{H}_{t+1}\mathbf{u}_t} \tag{2}$$

- Block BFGS update, choose $\mathbf{U}_t \in \mathbb{R}^{d\times k}$, which generalize equation (2)

$$\mathrm{BlockBFGS}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{U}_t) \triangleq$$
$$\mathbf{G}_t - \mathbf{G}_t\mathbf{U}_t\left(\mathbf{U}_t^\top\mathbf{G}_t\mathbf{U}_t\right)^{-1}\mathbf{U}_t^\top\mathbf{G}_t + \mathbf{H}_{t+1}\mathbf{U}_t\left(\mathbf{U}_t^\top\mathbf{H}_{t+1}\mathbf{U}_t\right)^{-1}\mathbf{U}_t^\top\mathbf{H}_{t+1}.$$

Choosing $\mathbf{U}_t \sim \mathcal{D}$ as a random matrix leads to the randomized block BFGS methods.

Quasi-Newton Methods for Convex Optimization

The non-asymptotic analysis of quasi-Newton methods was not built until the recent work by Rodomanov and Neterov. Then Ye et.al and Lin et.al sharper the convergence bound.

- Classical BFGS $\mathcal{O}\big((d\kappa/t)^t/2\big)$, $\mathcal{O}\big((\mathrm{e}^{(d\ln(\kappa)/t)}-1)^{t/2}\big)$.
- Classical SR1 $\mathcal{O}\big((d\ln(\kappa)/t)^t\big)$
- Greedy BFGS/SR1 $\mathcal{O}\big((1-1/(\kappa d))^{t(t-1)/2}\big)$
- Greedy/randomized BFGS/SR1 $\mathcal{O}\big((1-1/d)^{t(t-1)/2}\big)$

Quasi-Newton Methods

Although quasi-Newton methods is well-studied, there are lots of problems remain to solve.

- Block quasi-Newton methods with rank-k update are better in practice, can we give some theoretical results to show that they are indeed better?
- The quasi-Newton update are developed for convex optimization, where the object matrix is positive definite $\nabla^2 f(\mathbf{x}) \succ \mathbf{0}$.
  1. In minimax optimization, the Hessian is indefinite.
  2. In general non-linear equation , the Jacobian is even non-square!

Remain of the talk

- **Chengchang Liu**, Cheng Chen and Luo Luo. Symmetric Rank-k Methods. *arXiv Preprint*: 2303.16188.
- **Chengchang Liu**, Luo Luo. Quasi-Newton methods for saddle point problems. *NeurIPS*, 2022. *(Spotlight)*
- **Chengchang Liu**, Shuxian Bi, Luo Luo, John C.S. Lui. Partial-quasi-Newton methods: Efficient algorithms for minimax optimization problems with unbalanced dimensionality. *KDD*, 2022. *(Best Paper Award Runner-Up)*

1 Introduction

2 Symmetric Rank-k Methods

3 Squared-Quasi-Newton Methods

4 Partial-Quasi-Newton Methods

SR-k Methods

Consider the general form of update for convex optimization

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1} \nabla f(\mathbf{x}_t) \\ \mathbf{G}_{t+1} = \mathrm{Update}(\mathbf{G}_t), \end{cases}$$

- Gradient Descent: $\mathbf{G}_{t+1} = \mathbf{G}_t = L\mathbf{I} \rightarrow$ rank-0 update
- Newton Method: $\mathbf{G}_{t+1} = \nabla^2 f(\mathbf{x}_{t+1}) \rightarrow$ rank-d update
- Quasi-Newton Method: $\mathbf{G}_{t+1} = \mathrm{BFGS/SR1}(\mathbf{G}_t, \mathbf{H}_{t+1}, \mathbf{u}_t) \rightarrow$ rank-1 or rank-2 update

	Gradient-Descent	SR1	Newton Method
λ_{t+1}/λ_t	$(1 - 1/\kappa)$	$\mathcal{O}\left((1 - 1/d)^t\right)$	$\mathcal{O}(\lambda_t)$
Rank	0	1	d

SR-k Methods

What if we have rank-k update on $\mathbf{G}_t$ at each iteration? ($k \in [d]$)

	GD	SR1	?	Newton
λ_{t+1}/λ_t	$\left(1 - \frac{1}{\kappa}\right)$	$\mathcal{O}\left((1 - 1/d)^t\right)$	?	$\mathcal{O}(\lambda_t)$
Rank	0	1	$k \in [d]$	d

SR-k Methods

We proposed symmetric rank-k method!

	GD	SR1	SR-k	Newton
λ_{t+1}/λ_t	$\left(1-\frac{1}{\kappa}\right)$	$\mathcal{O}\big((1-1/d)^t\big)$	$\begin{cases}\mathcal{O}\big((1-k/d)^t\big), & k\in[d-1]\\ \mathcal{O}\big(\lambda_t\big), & k=d\end{cases}$	$\mathcal{O}(\lambda_t)$
Rank	0	1	$k\in[d]$	d

SR-k Update

We first propose SR-k update strategy to efficiently approximate the positive definite matrix $\mathbf{A}$.

$$\text{SR-}k(\mathbf{G}, \mathbf{A}, \mathbf{U}) \triangleq \mathbf{G} - (\mathbf{G} - \mathbf{A})\mathbf{U}(\mathbf{U}^\top(\mathbf{G} - \mathbf{A})\mathbf{U})^{-1}\mathbf{U}^\top(\mathbf{G} - \mathbf{A}).$$

- $\mathbf{A}, \mathbf{G} \in \mathbb{R}^{d\times d}$ are positive definite
- $\mathbf{U} \in \mathbb{R}^{d\times k}$ is some full rank matrix, typically, we choose $k \ll d$
- $\mathbf{AU} = [\mathbf{Au}_1, \cdots, \mathbf{Au}_k]$, the calculation of $\mathbf{Au}_i$ can be conducted in parallel.
- SR-k update reduce to the famous SR1 update when $k = 1$.

SR-k Update

- $\mathbf{U}$ can be conducted effeciently
  1. For randomized strategy, we sample each entry of $\mathbf{U}$ according to $\mathcal{N}(0,1)$ independently.
  2. For greedy strategy, We choose $\mathbf{U} = [\mathbf{e}_{i_1}; \cdots ; \mathbf{e}_{i_t}] \in \mathbb{R}^{d\times k}$, $i_1, \ldots, i_t$ are the indices for the largest k entries in the diagonal of $\mathbf{G} - \mathbf{A}$.
- If $\mathbf{A} \preceq \mathbf{G} \preceq \eta\mathbf{A}$, then

$$\mathbf{A} \preceq \mathbf{G}_+ = \text{SR-}k(\mathbf{G}, \mathbf{A}, \mathbf{U}) \preceq \eta\mathbf{A}.$$

- $\mathbf{G}_+ = \text{SR-}k(\mathbf{G}, \mathbf{A}, \mathbf{U})$ and $\tau_{\mathbf{A}}(\mathbf{G}) = \mathrm{tr}(\mathbf{G} - \mathbf{A})$, then

$$\mathbb{E}\left[\tau_{\mathbf{A}}(\mathbf{G}_+)\right] \leq \left(1 - \frac{k}{d}\right)\tau_{\mathbf{A}}(\mathbf{G}).$$

SR-k Methods

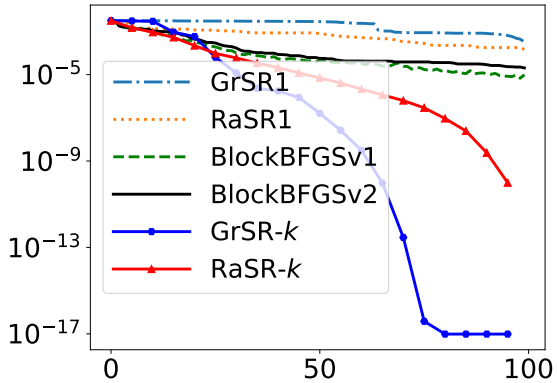
The SR-k methods use the following update scheme

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{G}_t^{-1}\nabla f(\mathbf{x}_t) \\ \tilde{\mathbf{G}}_t = (1 + M\|\mathbf{x}_{t+1} - \mathbf{x}_t\|_{\mathbf{x}_t})\mathbf{G}_t \\ \mathbf{G}_{t+1} = \text{SR-}k(\tilde{\mathbf{G}}_t, \nabla^2 f(\mathbf{x}_{t+1}), \mathbf{U}_t). \end{cases}$$

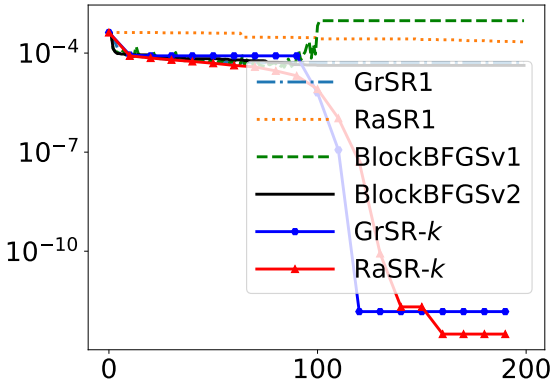
We have built explicit superlinear rate for SR-k which bridge the gap between the Newton methods ($k = d$) and quasi-Newton methods ($k = 1$)!

$$\lambda_{t+1}/\lambda_t = \begin{cases} \mathcal{O}\big((1 - k/d)^t\big), & k \in [d-1] \\ \mathcal{O}(\lambda_t), & k = d \end{cases} \tag{3}$$

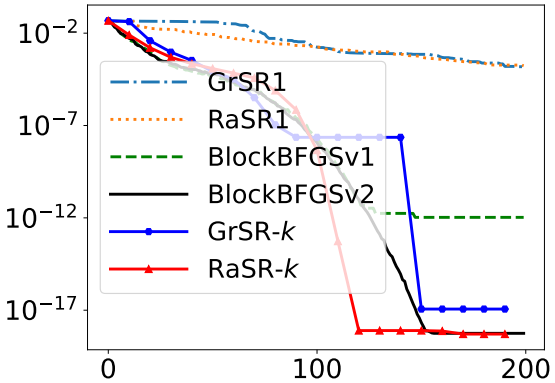
Experiments on Logistic Regression



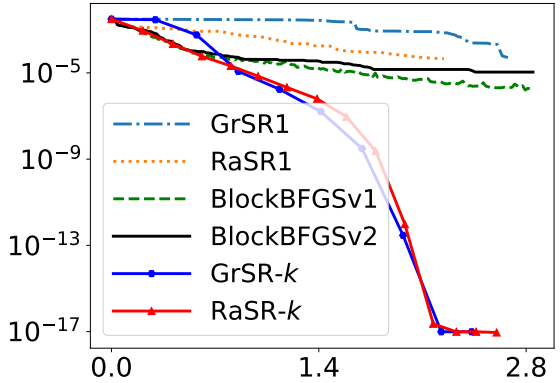
(a) a9a (iteration)



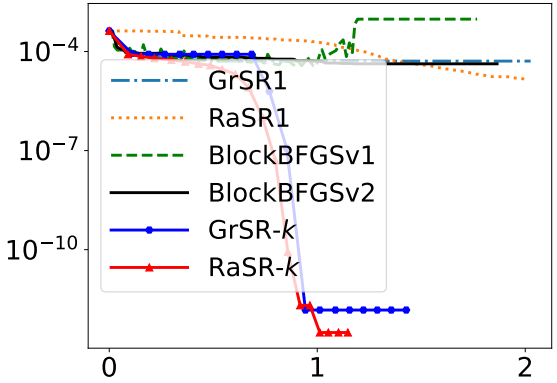
(b) madelon (iteration)



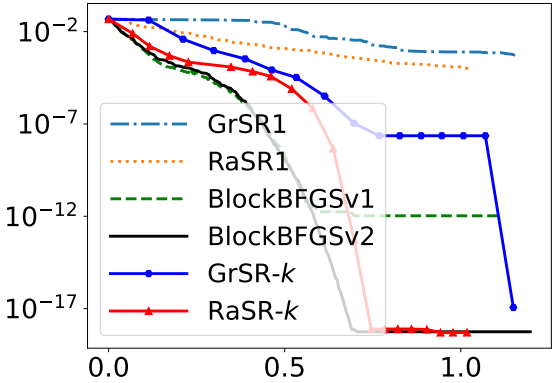
(c) mushroom (iteration)



(d) a9a (time)



(e) madelon (time)



(f) mushroom (time)

Here we choose $k = 5$.

Second-Order Optimization for Minimax Problems

Let's consider the minimax optimization

$$\min_{\mathbf{x}\in\mathbb{R}^{d_x}} \max_{\mathbf{y}\in\mathbb{R}^{d_y}} f(\mathbf{x},\mathbf{y}).$$

We denote $\mathbf{z} = [\mathbf{x}^\top; \mathbf{y}^\top]^\top$ and assume

- f is μ-strong convex on $\mathbf{x}$ and is μ-strong concave on $\mathbf{y}$
- $\|\nabla^2 f(\mathbf{z})\| \leq L$
- $\|\nabla^2 f(\mathbf{z}) - \nabla^2 f(\mathbf{z}')\| \leq L_2\|\mathbf{z} - \mathbf{z}'\|$

The Hessian of f

$$\nabla^2 f(\mathbf{x},\mathbf{y}) = \begin{bmatrix} \nabla^2_{\mathbf{xx}} f(\mathbf{x},\mathbf{y}) & \nabla^2_{\mathbf{xy}} f(\mathbf{x},\mathbf{y}) \\ \nabla^2_{\mathbf{yx}} f(\mathbf{x},\mathbf{y}) & \nabla^2_{\mathbf{yy}} f(\mathbf{x},\mathbf{y}) \end{bmatrix}$$

is indefinite!

1 Introduction

2 Symmetric Rank-k Methods

3 Squared-Quasi-Newton Methods

4 Partial-Quasi-Newton Methods

Squared-Quasi-Newton Methods

Denote $\mathbf{z} = (\mathbf{x}, \mathbf{y})$, $\mathbf{g}_t = \nabla f(\mathbf{x}_t, \mathbf{y}_t)$, $\hat{\mathbf{H}}_t = \nabla^2 f(\mathbf{x}_t, \mathbf{y}_t)$ and $\mathbf{H}_t = \hat{\mathbf{H}}_t^2$.

We rewrite the Newton update

$$\begin{aligned}
\mathbf{z}_{t+1} =& \mathbf{z}_k - \left(\nabla^2 f(\mathbf{z}_t)\right)^{-1} \nabla f(\mathbf{z}_t) \\
=& \mathbf{z}_k - \left(\nabla^2 f(\mathbf{z}_t)\right)^{-1} \left(\nabla^2 f(\mathbf{z}_t)\right)^{-1} \left(\nabla^2 f(\mathbf{z}_t) \nabla f(\mathbf{z}_t)\right) \\
=& \mathbf{z}_k - \left(\mathbf{H}_t^{-1}\right) \hat{\mathbf{H}}_t \mathbf{g}_t.
\end{aligned}$$

Some good news:

- The square of Hessian is $(2\kappa^2 L_2/L)$-Lipschitz continuous
- $\mathbf{H}_t = \hat{\mathbf{H}}_t^2 \succeq \mu^2 \mathbf{I}$
- $\hat{\mathbf{H}}_t \mathbf{g}_t$ and $\mathbf{H}_t \mathbf{g}_t$ can be computed efficiently

Approximating $\mathbf{H}_t$ leads to quasi-Newton for minimax!

Squared-Quasi-Newton Methods

Algorithm Squared-Quasi-Newton Methods (SQN)

1: **Input:** $\mathbf{z}_0 \in \mathbb{R}^n$, $\mathbf{G}_0 \succeq L^2\mathbf{I}$ and $M \geq 0$.

2: **for** $k = 0, 1 \ldots$

3: $\quad \mathbf{z}_{t+1} = \mathbf{z}_t - \mathbf{G}_t^{-1}\hat{\mathbf{H}}_t\mathbf{g}_t$

4: $\quad r_t = \|\mathbf{z}_{t+1} - \mathbf{z}_t\|$

5: $\quad \tilde{\mathbf{G}}_k = (1 + Mr_t)\mathbf{G}_t$

6: $\quad \mathbf{u}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

7: $\quad \mathbf{G}_{t+1} = \mathrm{SR1}(\tilde{\mathbf{G}}_t, \mathbf{H}_{t+1}, \mathbf{u}_t)$.

8: **end for**

Squared-Quasi-Newton Methods

SQN methods have local convergence rate

$$\lambda_{k+k_0} \leq \left(1 - \frac{1}{d}\right)^{\frac{k(k-1)}{2}} \left(1 - \frac{1}{4\kappa^2}\right)^{k_0} \left(\frac{1}{2}\right)^k \lambda_0$$

with probability $1 - \delta$, where $\kappa = L/\mu$, $d = d_x + d_y$,

$$\lambda_t = \|\nabla f(\mathbf{x}_t, \mathbf{y}_t)\|_2 \quad \text{and} \quad k_0 = \mathcal{O}\left(\left(d + \kappa^2\right) \ln\left(\frac{d\kappa}{\delta}\right)\right).$$

Squared-Quasi-Newton Methods

Let's consider an even more challege problem

$$\mathbf{g}(\mathbf{z}) = \mathbf{0}, \quad \mathbf{g} : \mathbb{R}^d \to \mathbb{R}^m$$

Let $\mathbf{H}(\mathbf{z})$ be Jacobian of $\mathbf{g}$ at $\mathbf{z}$. $\mathbf{H}(\mathbf{z}) \in \mathbb{R}^{m \times d}$ is even not square!
We suppose

- there exist $\mathbf{z}^*$ such that $\mathbf{H}(\mathbf{z}^*)$ is non-singular;
- $\mathbf{H}(\cdot)$ is L_2-Lipschitz continuous.

The LM methods

$$\mathbf{z}_{t+1} = \mathbf{z}_t - (\mathbf{H}(\mathbf{z}_t)^\top \mathbf{H}(\mathbf{z}_t) + \lambda_t \mathbf{I})^{-1} \mathbf{H}(\mathbf{z}_t)^\top \mathbf{g}(\mathbf{z}_t).$$

The quasi-Newton methods for nonlinear equations

$$\begin{cases} \mathbf{z}_{t+1} = \mathbf{z}_t - \mathbf{G}_t{}^{-1} \mathbf{H}(\mathbf{z}_t)^\top \nabla f(\mathbf{x}_t) \\ \tilde{\mathbf{G}}_t = (1 + M\|\mathbf{z}_{t+1} - \mathbf{z}_t\|)\mathbf{G}_t \\ \mathbf{G}_{t+1} = \text{SR-}k(\mathbf{G}_t, \mathbf{H}(\mathbf{z}_{t+1})^\top \mathbf{H}(\mathbf{z}_t), \mathbf{U}_t). \end{cases}$$

Machine Learning Application

- **AUC maximization** aims to find the classifier $\mathbf{w} \in \mathbb{R}^m$ on the training set $\{\mathbf{a}_i, b_i\}_{i=1}^n$ where $\mathbf{a}_i \in \mathbb{R}^m$ and $b_i \in \{+1, -1\}$.

$$\min_{\mathbf{x} \in \mathbb{R}^{m+2}} \max_{y \in \mathbb{R}} f(\mathbf{x}, y) \triangleq \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x}, \mathbf{y}; \mathbf{a}_i, b_i, \lambda),$$

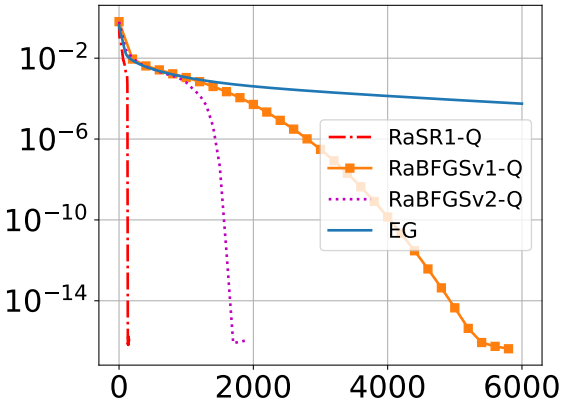
  where $\mathbf{x} = [\mathbf{w}; u; v] \in \mathbb{R}^{m+2}$, $d_x = m + 2$ and $d_y = 1$.

- **Fairness-aware machine learning** aims to make the input variable $c_i \in \mathbb{R}$ unbiased on the training set $\{\mathbf{a}_i, b_i, c_i\}_{i=1}^n$.

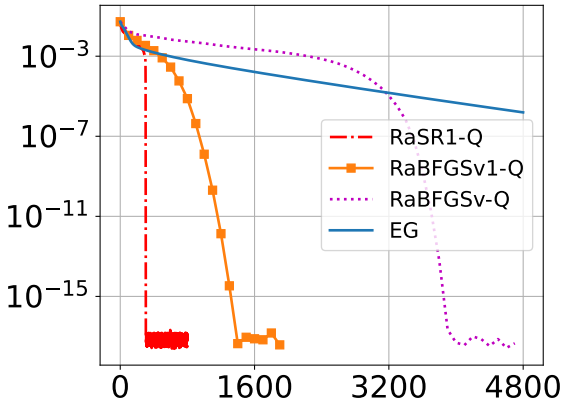
$$\min_{\mathbf{x} \in \mathbb{R}^m} \max_{y \in \mathbb{R}} \frac{1}{n} \sum_{i=1}^n (l_1(b_i \mathbf{a}_i^\top \mathbf{x}) - \beta l_2(c_i y \mathbf{a}_i^\top \mathbf{x})) + \lambda \|\mathbf{x}\|^2 - \gamma y^2,$$

  where $d_x = m$, $d_y = 1$.

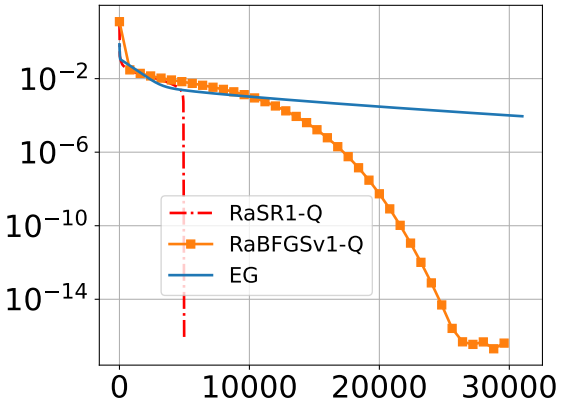
Experiments on AUC maximization



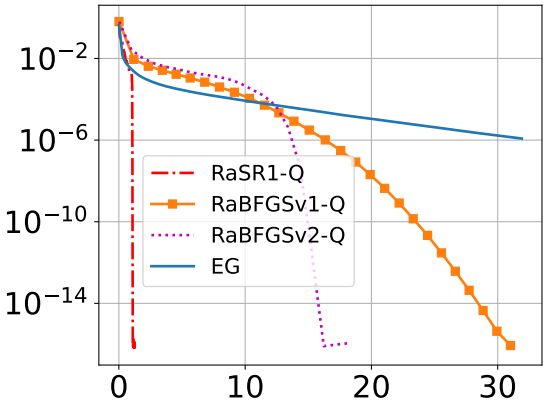
(a) a9a (iteration)



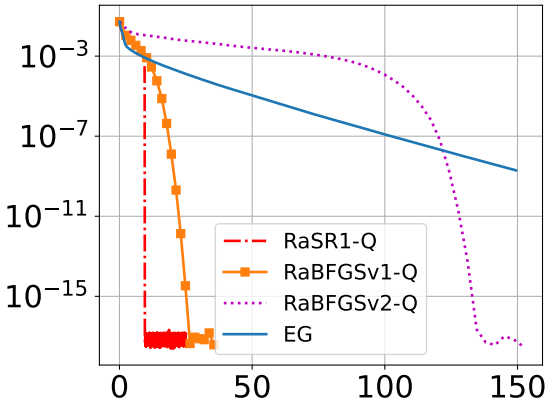
(b) w8a (iteration)



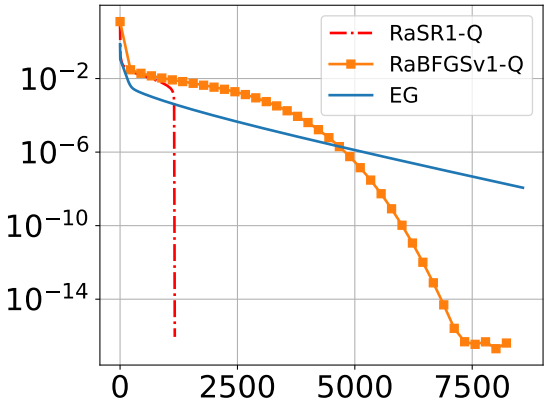
(c) sido0 (iteration)



(d) a9a (time)

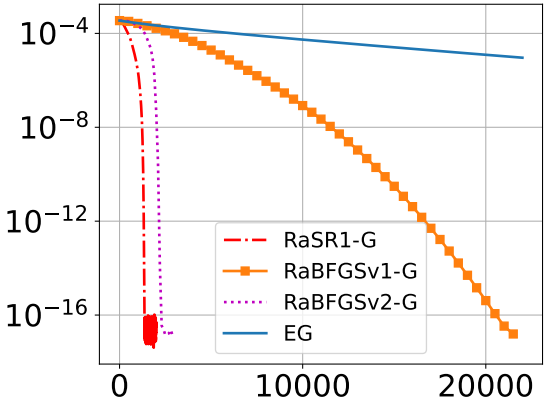


(e) w8a (time)

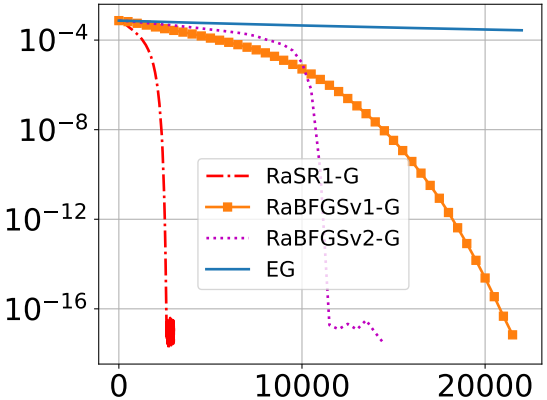


(f) sido0 (time)

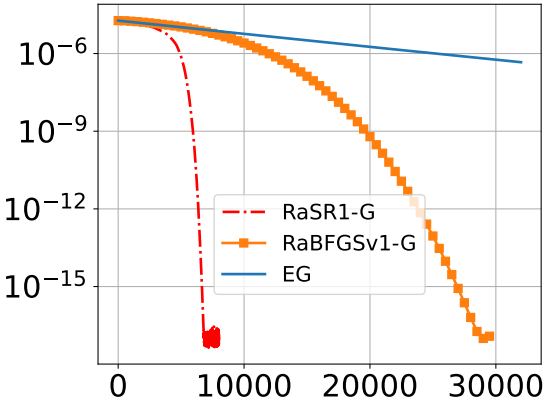
Experiments on Adversarial Debiasing



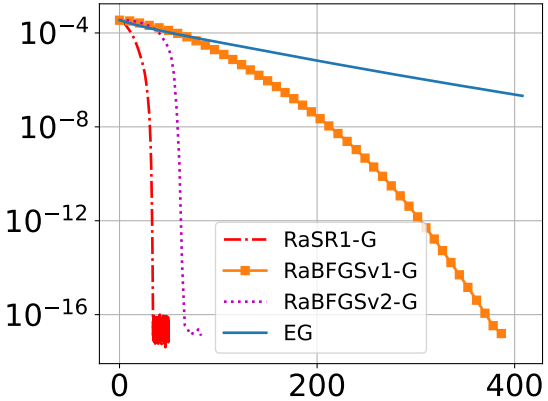
(a) adults (iteration)



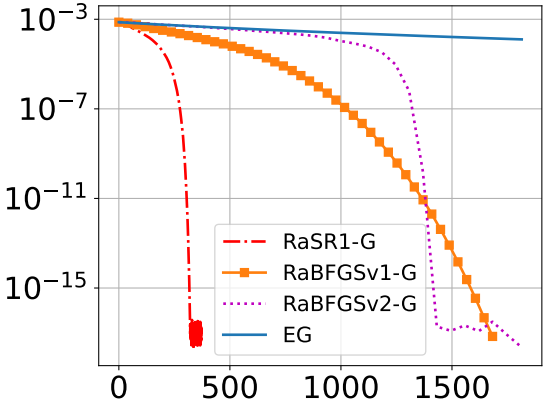
(b) school (iteration)



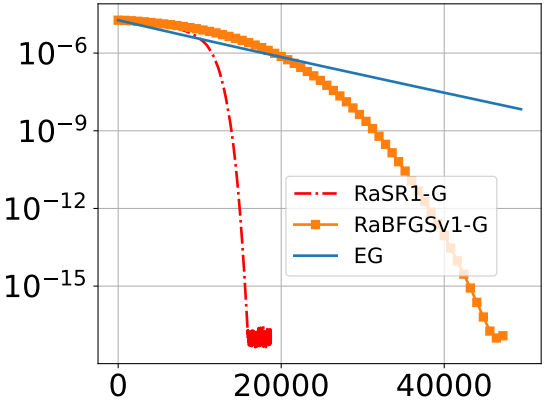
(c) bank (iteration)



(d) adults (time)



(e) school (time)



(f) bank (time)

1 Introduction

2 Symmetric Rank-k Methods

3 Squared-Quasi-Newton Methods

4 Partial-Quasi-Newton Methods

Some Popular Machine Learning Applications

The weakness of squared-quasi-Newton methods:

- It does not differ $\mathbf{x}$ and $\mathbf{y}$.
- The greedy method could be so expensive.
- The square of $\nabla^2 f(\mathbf{x}, \mathbf{y})$ has worse condition number.

There is "unbalanced dimensionality" in real applications.

- For AUC maximization, we have $d_x \gg d_y = 1$.
- Computing the partial Hessian could be very cheap.

Partial-Quasi-Newton Methods

Partial-Quasi-Newton methods construct the Hessian estimator by

$$\nabla^2 f(\mathbf{x}_t, \mathbf{y}_t) = \begin{bmatrix} \nabla^2 f_{\mathbf{xx}}(\mathbf{x}_t, \mathbf{y}_t) & \nabla^2 f_{\mathbf{xy}}(\mathbf{x}_t, \mathbf{y}_t) \\ \nabla^2 f_{\mathbf{yx}}(\mathbf{x}_t, \mathbf{y}_t) & \nabla^2 f_{\mathbf{yy}}(\mathbf{x}_t, \mathbf{y}_t) \end{bmatrix}$$
$$\approx \begin{bmatrix} \mathbf{G}_t & \nabla^2 f_{\mathbf{xy}}(\mathbf{x}_t, \mathbf{y}_t) \\ \nabla^2 f_{\mathbf{yx}}(\mathbf{x}_t, \mathbf{y}_t) & \nabla^2 f_{\mathbf{yy}}(\mathbf{x}_t, \mathbf{y}_t) \end{bmatrix},$$

where $\mathbf{G}_t$ is obtained by Broyden family update.

Partial-Quasi-Newton Methods

Algorithm Partial-Quasi-Newton Methods (PQN)

Input: $\mathbf{G}_{\mathbf{xx},0}$, $\mathbf{z}_0$ and $M \geq 0$
$\mathbf{G}_0^{-1} = \mathrm{Inverse}\big(\mathbf{G}_{\mathbf{xx},0}^{-1}, \mathbf{H}_{\mathbf{xy},0}, \mathbf{H}_{\mathbf{yy},0}\big)$
for $k = 0, 1, \ldots$
    $\mathbf{z}_{t+1} = \mathbf{z}_t - \mathbf{G}_t^{-1}\mathbf{g}(\mathbf{z}_t)$
    $r_t = \langle \mathbf{x}_{t+1} - \mathbf{x}_t, \mathbf{P}(\mathbf{z}_t)(\mathbf{x}_{t+1} - \mathbf{x}_t)\rangle^{1/2} + \|\mathbf{y}_{t+1} - \mathbf{y}_t\|$
    $\tilde{\mathbf{G}}_{\mathbf{xx},k} = (1 + Mr_t)\mathbf{G}_{\mathbf{xx},k}$
    Choose $\mathbf{u}_t$ by greedy/random method
    $\mathbf{G}_{\mathbf{xx},k+1} = \mathrm{SR1}\big(\tilde{\mathbf{G}}_{\mathbf{xx},k}, \mathbf{H}_{\mathbf{xx}}(\mathbf{z}_{t+1}), \mathbf{u}_t\big)$
    $\mathbf{G}_{t+1}^{-1} = \mathrm{Inverse}\big(\mathbf{G}_{\mathbf{xx},k+1}^{-1}, \mathbf{H}_{\mathbf{xy}}(\mathbf{z}_{t+1}), \mathbf{H}_{\mathbf{yy}}(\mathbf{z}_{t+1})\big)$
end for

Partial-Quasi-Newton Methods

We measure the convergence by

$$\gamma_t = \left(\nabla_{\mathbf{x}} f(\mathbf{x}_t, \mathbf{y}_t)^\top \mathbf{S}_t^{-1} \nabla_{\mathbf{x}} f(\mathbf{x}_t, \mathbf{y}_t)\right)^{1/2} + \frac{2}{\sqrt{\mu}} \left\|\nabla_{\mathbf{y}} f(\mathbf{x}_t, \mathbf{y}_t)\right\|_2 .$$

For SR1-type method, we have

$$\gamma_{k_0+k} \leq \left(1 - \frac{1}{d}\right)^{\frac{k(k-1)}{2}} \left(\frac{1}{2}\right)^k \left(1 - \frac{1}{4\kappa}\right)^{k_0} \gamma_0$$

with probability $1 - \delta$, where

$$\mathbf{S}_t = \nabla^2_{\mathbf{xx}} f(\mathbf{x}_t, \mathbf{y}_t) - \nabla^2_{\mathbf{xy}} f(\mathbf{x}_t, \mathbf{y}_t) \left(\nabla^2_{\mathbf{yy}} f(\mathbf{x}_t, \mathbf{y}_t)\right)^{-1} \nabla^2_{\mathbf{yx}} f(\mathbf{x}_t, \mathbf{y}_t)$$

and $k_0 = \mathcal{O}\big(\max\{d, \kappa\} \ln(d\kappa)\big)$.

Partial-Quasi-Newton Methods

For the Optimal First-Order method, we have

$$\rho_{k+k_0} \leq \left(1 - \frac{1}{4\varkappa}\right)^{k+k_0} \rho_0 \quad \text{where} \quad \rho_k = \|\mathbf{z}_k - \mathbf{z}^*\|^2.$$

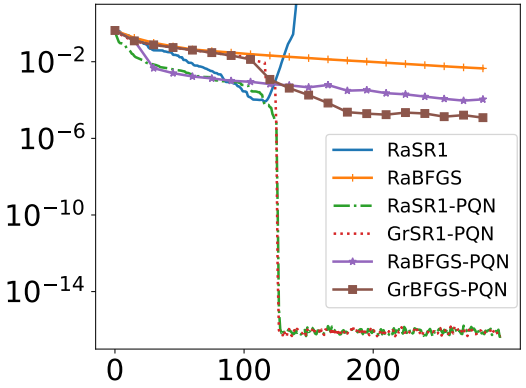
For the Squared-Quasi-Newton method, we have

$$\lambda_{k+k_0} \leq \left(1 - \frac{1}{d}\right)^{\frac{k(k-1)}{2}} \left(1 - \frac{1}{4\varkappa^2}\right)^{k_0} \left(\frac{1}{2}\right)^k \lambda_0 \quad \text{where} \quad \lambda_k = \|\nabla f(\mathbf{z}_k)\|.$$

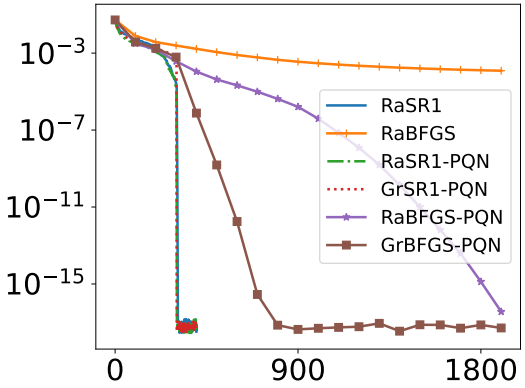
For the Partial-Quasi-Newton method, we have

$$\gamma_{k_0+k} \leq \left(1 - \frac{1}{d}\right)^{\frac{k(k-1)}{2}} \left(\frac{1}{2}\right)^k \left(1 - \frac{1}{4\varkappa}\right)^{k_0} \gamma_0.$$

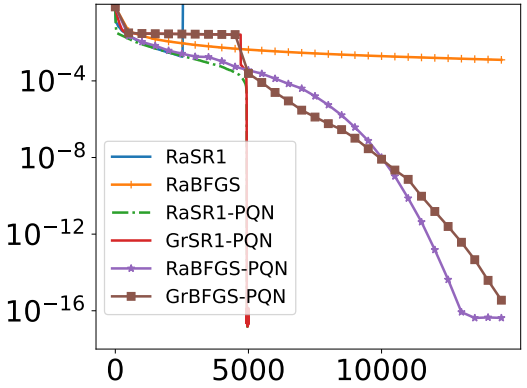
Experiments on AUC Maximization



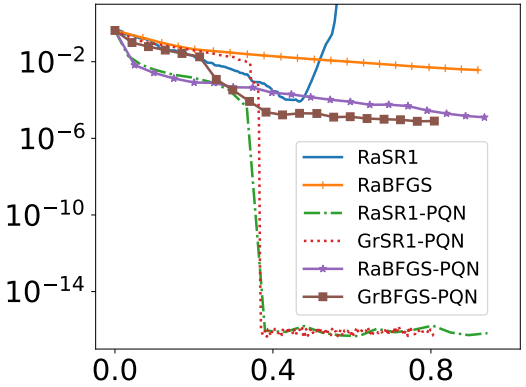
(a) a9a (iteration)



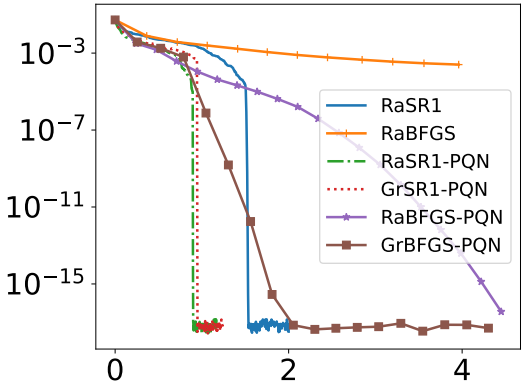
(b) w8a (iteration)



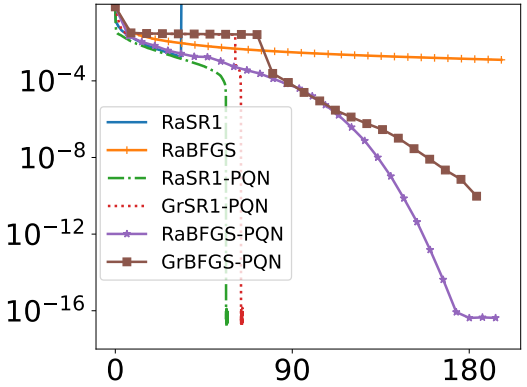
(c) sido0 (iteration)



(d) a9a (time)

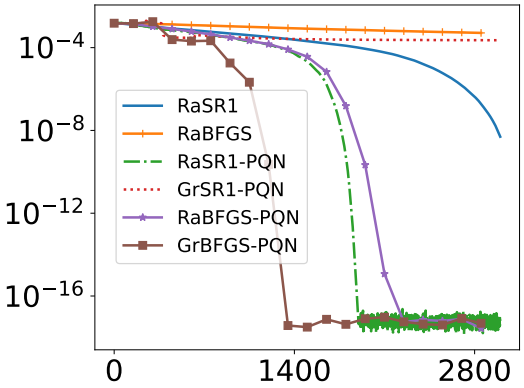


(e) w8a (time)

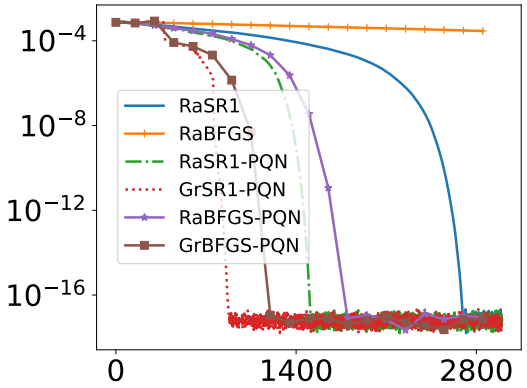


(f) sido0 (time)

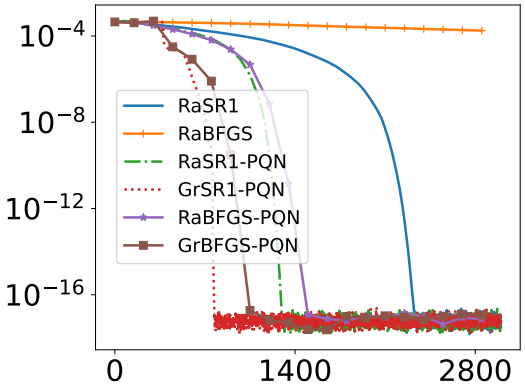
Experiments on Adversarial Debiasing



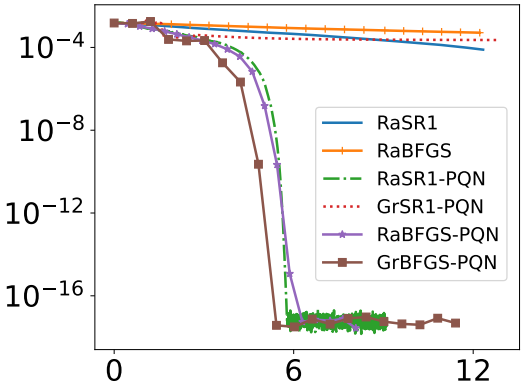
(a) $T = 20$ (iteration)



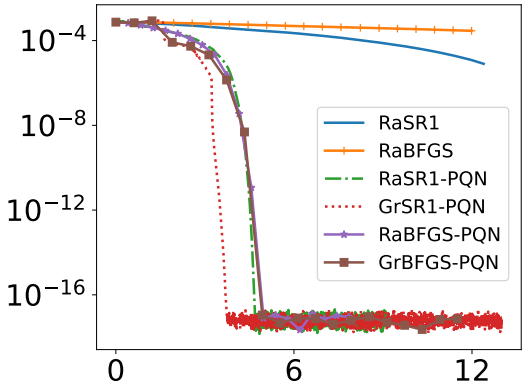
(b) $T = 200$ (iteration)



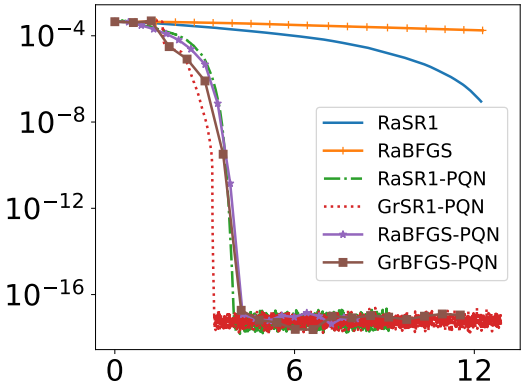
(c) $T = 2000$ (iteration)



(d) $T = 20$ (time)



(e) $T = 200$ (time)



(f) $T = 2000$ (time)

Results on "school" dataset with different iteration numbers of warm-up

Some Open Problems

The theory of quasi-Newton is not perfect:

1. How to match the global convergence of first-order methods?
2. What is the lower bound for superlinear convergence?
3. What is the local convergence of limited-memory quasi-Newton?